Go 秒杀系统

Redis 预扣 + MySQL 兜底的综合实战

Go 秒杀系统

一、设计文档：Seckill Service

§ 0 文档信息

§ 1 Context and Scope（背景与范围）

§ 2 Goals and Non-Goals（目标与非目标）

§ 3 Overview（总体方案）

§ 4 The Actual Design（详细设计）

§ 5 Alternatives Considered（备选方案）

§ 6 Cross-cutting Concerns（横切关注点）

§ 7 Degree of Constraint（约束程度）

§ 8 设计点 → 知识点对照

二、代码实现

项目结构与使用

sql/schema.sql

internal/cache/lua/deduct.lua

internal/cache/lua/rollback.lua

internal/cache/lua/ratelimit.lua

internal/cache/keys.go

internal/cache/cache.go

internal/store/mysql.go

internal/mq/stream.go

internal/service/seckill.go

internal/api/handler.go

cmd/seckill/main.go

internal/config/config.go

internal/model/model.go

internal/idgen/snowflake.go

internal/metrics/metrics.go

go.mod

> 使用说明：本篇从 Redis & MySQL 笔记拆出，包含设计文档和对应的 Go 代码。代码与仓库根目录 `seckill/` 一致（https://github.com/LosAveRoad/interview-prep/tree/main/seckill）。

> **知识点标注**：【R-N】链到 Redis 笔记 第 N 个知识点（1–29），【M-N】链到 MySQL 笔记 第 N 个知识点（1–38，在 MySQL 笔记内单独编号）。例如 【R-29】 是「秒杀场景与防超卖」，【M-16】 是「RR 下行锁加锁规则」。

Go 秒杀系统

一、设计文档：Seckill Service

§0 文档信息

项	内容
标题	Seckill Service：基于 Redis 预扣 + MySQL 兜底的 Go 秒杀服务
作者	Lisa
状态	草案（学习用示例，未经生产验证）
评审人	—

§1 Context and Scope（背景与范围）

电商平台需要支持「限时、限量、低价」的秒杀活动：某个商品在约定时刻开售，库存几十到几千件，而同一时刻涌入的请求可能比库存多几个数量级。这类流量有三个特点：

- **读多写少、瞬时峰值**：开售前大量刷新详情页，开售瞬间集中提交下单；
- **绝大多数请求注定失败**：库存远小于请求数，系统价值在于**又快又准地拒绝**；
- **写集中在一行**：所有下单都要扣同一个商品的库存，是典型的**热点行**。

若直接让请求打到 MySQL，`UPDATE ... SET stock = stock - 1` 会在同一行记录锁上排队【M-15】【M-16】，连接池被占满，进而拖垮其他业务。

范围：本服务负责活动查询、秒杀资格校验、库存扣减、订单创建与结果查询。商品管理、支付、物流、用户登录由其他系统负责，本服务只依赖其接口或数据。

现有技术栈：Go 后端，MySQL 8.0（InnoDB）作为业务主库，Redis 6.2+（主从 + 哨兵，或 Cluster）作为缓存。

§2 Goals and Non-Goals（目标与非目标）

Goals

1. **不超卖**：任何情况下，成功订单数 ⩽ 活动库存。
2. **一人一单**：同一用户在同一活动最多一个有效订单。
3. **扛住峰值**：峰值请求由 Redis 与限流承接，MySQL 写入速率只与库存量和消费速度相关，与请求量无关。
4. **数据最终一致**：Redis 预扣结果与 MySQL 订单、库存最终一致，并有对账发现不一致。

5. **可观测、可降级**：关键指标可监控，Redis/MQ 故障时可快速止损。

Non-Goals

- **支付与超时未支付取消**：不在本文范围。后续可用延迟队列（ZSet 或 MQ 延迟消息）实现超时关单并回补库存【R-28】。
- **一单多件、购物车合并下单**：每次固定购买 1 件。
- **验证码、风控模型、设备指纹**：只预留令牌校验入口，具体风控由风控系统负责。
- **少卖为零**：允许极端故障下少卖（库存未卖完），但绝不超卖。这是有意识的取舍，见第 6 节。
- **多活/异地容灾**。

§ 3 Overview（总体方案）

核心思路：**层层削减流量，把并发控制前移到 Redis，把最终正确性交给 MySQL**。

1. **拦截**：CDN/静态页挡掉详情页刷新；网关与实例内令牌桶限流，用户级计数限流【R-28】；
2. **资格**：开售后下发一次性秒杀令牌，令牌用 `GETDEL` 消费，接口地址对脚本不可预知；
3. **预扣**：一段 Lua 脚本原子完成「判重 → 判库存 → 扣减 → 记录已购 → 投递订单消息 → 写排队结果」【R-27】【R-29】；
4. **异步落库**：消费者从 Redis Stream 取消息，在 MySQL 事务中插入订单并做条件扣减【M-10】【M-18】；
5. **兜底与对账**：唯一索引保证一人一单【M-9】，`WHERE stock > 0` 保证不超卖，定时对账 Redis 与 MySQL。

用户感知的流程：点击「抢购」→ 立即返回「排队中」（HTTP 202）→ 前端轮询结果 → 「抢购成功，请支付」或「已售罄」。

§ 4 The Actual Design（详细设计）

4.1 系统架构图



```
| (4) Stream 消费者 |
+------------------+
      |  Lua 预扣 / 令牌 / 限流计数 / 结果 / 活动缓存
      v
+-------------------------------------------------------------+
| Redis（主从+哨兵 或 Cluster）                                 |
| sk:{aid}:stock    sk:{aid}:users    sk:{aid}:orders(Stream) |
| sk:{aid}:info     sk:{aid}:result:uid    sk:{aid}:token:uid |
+-------------------------------------------------------------+
      |  XREADGROUP（消费者组）
      v
+-------------------------------------------------+
| 订单消费者（与 API 同进程，也可拆成独立服务）     |
| 事务：INSERT 订单 -> UPDATE 库存 WHERE stock > 0 |
+-------------------------------------------------+
      |
      v
+-------------------------------------------------+
| MySQL 主库（InnoDB）  --binlog--> 从库（读 / 对账） |
| seckill_activity        seckill_order           |
+-------------------------------------------------+

  后台任务：活动预热（定时扫描即将开始的活动）、Redis/MySQL 对账、死信处理
```

4.2 API

所有接口返回 JSON，用户身份由网关校验 JWT 后以 `X-User-ID` 头注入（示例代码直接读该头）。

方法与路径	说明	成功	主要错误
`GET /api/v1/activities/{id}`	活动详情（走缓存）	200 活动 JSON	404
`POST /api/v1/activities/{id}/token`	开售后获取一次性秒杀令牌（60 s 有效）	200 `{"token"}`	400 未开始/已结束，429
`POST /api/v1/activities/{id}/seckill`	提交秒杀，Body `{"token"}`	**202** `{"status":"queued","order_id"}`	403 令牌无效，409 售罄/重复，429 限流
`GET /api/v1/activities/{id}/result`	轮询结果	200 `{"status":"queued/success/failed/none"}`	—
`POST /admin/activities/{id}/preheat`	手动预热（`X-Admin-Token`）	200	401，404
`GET /debug/vars`	expvar 指标	200	—

设计说明：

- 秒杀接口返回 **202 Accepted** 而不是 201：此时订单只是进入队列，尚未落库。
- 订单号在**预扣时由雪花算法生成**并随消息传递【M-31】，消费者用它做幂等判断，用户也能立即拿到单号。
- 金额字段以字符串返回，与 MySQL `DECIMAL` 对应，避免浮点误差【M-34】。

4.3 数据存储

MySQL 表结构（完整 DDL 见第二节 `sql/schema.sql`）

表	关键字段	索引	设计依据
`seckill_activity`	`id` 自增主键、`price DECIMAL(10,2)`、`total_stock`、`stock INT UNSIGNED`、`start_time`、`end_time`、`status`、`version`	PK(`id`)；`idx_status_start(status, start_time)`	自增主键顺序写入减少页分裂【M-4】【M-5】；预热任务按「状态等值 + 时间范围」查询，符合最左前缀【M-7】；`version` 用于后台编辑的乐观锁【M-18】
`seckill_order`	`id` 雪花 ID、`activity_id`、`user_id`、`goods_id`（冗余）、`price`、`status`	PK(`id`)；**`uk_user_activity(user_id, activity_id)`**；`idx_activity_status(activity_id, status)`	唯一索引兜底一人一单【M-9】【M-37】；`user_id` 在前，同时支撑「我的订单」查询；对账 `COUNT(*)` 可走二级索引覆盖【M-6】【M-26】；冗余 `goods_id` 避免 JOIN（反范式）【M-36】

- 两表均用 InnoDB：需要事务、行锁与崩溃恢复【M-2】【M-19】。
- 雪花 ID 趋势递增，作为主键插入时基本是顺序追加，同时为将来按 `user_id` 分库分表预留全局唯一 ID【M-31】。

Redis Key 设计

Key	类型	内容	TTL	依据
`sk:{aid}:stock`	String	剩余可预扣库存	活动结束 + 1 天	预热时 `SETNX` 写入，防止重复预热覆盖【R-29】
`sk:{aid}:users`	Set	已抢到的用户 ID	同上	`SISMEMBER` O(1) 判重；万级成员时注意大 key【R-3】【R-20】
`sk:{aid}:orders`	Stream	下单消息 `uid/oid/ts`	`MAXLEN ~` 截断	消费者组 + ACK，可靠消息队列【R-3】
`sk:{aid}:info`	String(JSON)	活动详情缓存	10 min + 随机 0~120 s	Cache Aside、TTL 抖动防雪崩【R-19】【R-25】
`sk:{aid}:result:{uid}`	String	`queued:oid` / `success:oid` / `failed:原因`	30 min	供前端轮询，减少查库
`sk:{aid}:token:{uid}`	String	一次性令牌	60 s	`GETDEL` 原子取出并删除
`sk:rl:{uid}:{秒}`	String	用户每秒请求计数	2 s	固定窗口限流【R-28】
`sk:dlq`	Stream	死信消息	人工处理后清理	可靠性兜底

- **Hash Tag**：活动相关 key 都含 `{aid}`，Cluster 只对花括号内的内容计算 CRC16 槽位，所以它们落在同一个哈希槽，多 key 的 Lua 脚本才能执行【R-15】。代价是一个活动的流量集中在

一个分片上，见热点讨论（4.4.7）。

- **命名**：`业务:{实体}:属性`，便于 `SCAN` 按前缀排查（禁止 `KEYS`）【R-21】。

4.4 关键流程

4.4.1 活动预热

1. 定时任务每 30 s 查询「已发布、`start_time` ≤ 现在 + 10 min、未结束」的活动（命中 `idx_status_start`）【M-7】；
2. `SETNX sk:{aid}:stock <stock>`，已存在则跳过，**不覆盖进行中的库存**；
3. `XGROUP CREATE sk:{aid}:orders order-creator 0 MKSTREAM`，忽略 `BUSYGROUP`；
4. 每个活动启动一个 Stream 消费协程（同一进程只启动一次）。

预热把库存提前放进内存，开售瞬间就不会出现缓存未命中导致的回源击穿【R-18】。

4.4.2 秒杀主流程（库存预扣 Lua 原子扣减）

```
请求
  |
  v
实例令牌桶 -------------------- 拿不到令牌 -----> 429
  |
  v
本地售罄标记? ----------------- 是 -------------> 409 售罄
  | 否
  v
用户级限流 (Lua INCR+EXPIRE) -- 超限 -----------> 429
  |
  v
活动时间校验 (缓存) ----------- 未开始/已结束 --> 400
  |
  v
GETDEL 令牌校验 --------------- 失败 -----------> 403
  |
  v
雪花算法生成 order_id
  |
  v
+-------------------------------------------------+
| deduct.lua (整段原子执行)                       |
| SISMEMBER users uid  == 1  -> 返回 -2 (重复)    |
| GET stock == nil           -> 返回 -3 (未预热)  |
| stock <= 0                 -> 返回 -1 (售罄)    |
| DECR stock;  SADD users uid                     |
| XADD orders uid oid ts                          |
| SET result:uid "queued:oid" EX ttl              |
| 返回剩余库存                                    |
+-------------------------------------------------+
  |
  v
202 queued + order_id; 剩余为 0 时设置本地售罄标记
```

- **为什么用 Lua**：「判断 + 扣减 + 记录」是**读-判断-写**的复合操作，分成多条命令在并发下会出现两个请求都读到 `stock=1` 的竞态。Redis 单线程执行命令【R-1】，Lua 脚本执行期间不会插

入其他命令，相当于一个原子块【R-27】。`MULTI/EXEC` 做不到「根据读到的值决定是否写」，`WATCH` 乐观重试在热点下冲突率极高，所以选 Lua。

- **消息投递放进同一个脚本**：扣减与入队要么都发生、要么都不发生，消除「扣了库存但消息没发出去」的窗口。这是本方案选 Redis Stream 而不是外部 MQ 的主要原因（见第 5 节）。
- **脚本要短**：脚本内只有 O(1) 操作，避免阻塞 Redis【R-22】；go-redis 的 `Script.Run` 先 `EVALSHA`，`NOSCRIPT` 时自动回退 `EVAL`。
- **本地售罄标记**：Redis 返回售罄后，本实例用 `sync.Map` 记住，后续请求连 Redis 都不访问。回滚归还库存时只能清除本实例的标记，其他实例会多拒绝一段时间，这属于「少卖」，可接受；若要精确，可用 Redis Pub/Sub 广播清除。

4.4.3 防超卖（三道防线）

防线	手段	失效场景
1. Redis 预扣	Lua 中 `stock <= 0` 直接拒绝【R-27】【R-29】	主从切换丢失最近写入：主节点异步复制，宕机时新主可能缺少部分 `DECR`【R-13】【R-14】【R-16】；AOF `everysec` 宕机最多丢约 1 s 写入【R-9】
2. MySQL 条件更新	`UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`，影响 0 行即失败	— 这是最终防线
3. 对账	定时比对 Redis 库存 + 已购人数、MySQL 订单数	只发现问题，不阻止

- 条件更新是**当前读**，读的是最新已提交版本并加 X 型记录锁（`id` 为主键等值，RR 下退化为记录锁，不加间隙锁）【M-14】【M-15】【M-16】，所以并发事务对同一行串行执行，不会出现「两个事务都看到 stock=1」。
- 不能用「先 `SELECT stock`（快照读）再 `UPDATE stock = 读到的值 - 1`」：快照读读的是 MVCC 版本【M-13】，并发下会丢失更新。
- `stock` 定义为 `INT UNSIGNED`，即使逻辑出错，减到负数也会报错（严格 SQL 模式下），多一层保险。

4.4.4 一人一单

- **Redis 层**：Lua 内 `SISMEMBER` + `SADD`，与扣减在同一原子块中，并发的两个请求只有一个能通过。
- **MySQL 层**：`UNIQUE KEY uk_user_activity(user_id, activity_id)`；重复插入返回错误码 1062，代码映射为 `ErrDuplicateOrder`【M-9】【M-38】。
- **不用分布式锁**：按用户加锁（`SET NX PX`）也能做到，但多一次网络往返，还要处理锁过期与误删【R-23】【R-24】；Lua 的 Set 判重已经足够。

4.4.5 异步下单（Redis Stream 消息队列）

```
deduct.lua XADD --> sk:{aid}:orders --XREADGROUP ">"--> 消费者 --> MySQL 事务
                          |                                  |
                          |        成功 / 永久错误: XACK <--+
                          |        临时错误: 不 ACK，留在 PEL
                          v
reclaimLoop: XPENDING(idle > 30s) -> XCLAIM -> 重新处理
             投递次数 >= MaxRetry -> 写 sk:dlq -> 回滚预扣 -> XACK
```

消费者事务（`store.CreateOrder`）：

1. `BEGIN`
2. `INSERT INTO seckill_order ...`：若唯一键冲突（1062）立即返回，**不触碰热点行**；
3. `UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`：放在**最后一步**，热点行 X 锁从加锁到提交的时间最短（InnoDB 行锁在事务提交或回滚时才释放，所以越晚加锁、持锁越短）【M-15】【M-17】；
4. `COMMIT`：redo log 与 binlog 通过两阶段提交保证一致，从库与对账读到的数据可信【M-19】【M-21】【M-22】【M-23】。

结果处理（消息至少投递一次，所以消费必须幂等）：

事务结果	含义	处理
成功	正常下单	写结果 `success:oid`，XACK
1062 且已有订单 `id` 等于消息中的 `oid`	同一条消息重复投递	视为成功（幂等），XACK
1062 且订单 `id` 不同	用户已有其他订单（如 Redis 数据丢失后又抢到一次）	回滚预扣并**归还** Redis 库存，结果 `failed:repeat`
影响 0 行	MySQL 已售罄（Redis 与 DB 不一致）	回滚预扣但**不归还**库存，结果 `failed:sold_out`
数据库超时等	临时故障	不 ACK，等待重试；超过 `MaxRetry` 进入死信并回滚

- `rollback.lua` 以 `SREM` 的返回值作为幂等开关：只有用户仍在已购集合中才归还库存，重复回滚不会多加库存。
- 每个活动一个 Stream、一个消费协程：保证 Cluster 下单 key 操作，并且自然把数据库写入速率限制在「消费者数 × 单条事务耗时」，实现**削峰填谷**。

4.4.6 缓存一致性（活动详情）

- **读**：Cache Aside——先读缓存，未命中查库再回写【R-25】。
  - 不存在的活动缓存空值 `<nil>` 1 分钟，防**穿透**【R-17】（活动 ID 量级小，不必上布隆过滤器）；
  - 未命中时用 `singleflight` 合并同一 key 的并发回源，防**击穿**【R-18】；

  - TTL 10 分钟 + 0~120 秒随机抖动，防**雪崩**【R-19】；
  - Redis 故障时直接查库，由前置限流保护 MySQL。
- **写**（后台修改标题/价格）：**先更新数据库，再删除缓存**，删除失败重试 3 次，仍失败靠 TTL 兜底【R-25】。更新时 `WHERE version = ?` 乐观锁，防止两个运营同时编辑互相覆盖【M-18】。
  - 不选「先删缓存再更新库」：并发读会把旧值回填缓存；
  - 进一步可订阅 binlog（如 Canal）异步删除缓存，与业务代码解耦【R-26】【M-21】。
- **库存不走 Cache Aside**：库存是「Redis 为准做预扣、MySQL 为准做最终结算」的两套计数，靠消息与对账保证最终一致，不能用「删缓存」的方式同步，否则会把正在扣减的库存重置。

4.4.7 限流与热点

层级	算法	位置	依据
网关	IP/用户维度限流、黑名单	网关（不在本服务代码中）	—
实例级	令牌桶 `golang.org/x/time/rate`，拿不到令牌立即 429，不排队	进程内存	【R-28】
用户级	固定窗口：`INCR` + 首次 `EXPIRE`，同一 Lua 脚本内执行	Redis	【R-27】【R-28】
下游写入	消费者数量固定，天然限速	Stream 消费	—

- 用户级计数的 INCR 和 EXPIRE 必须原子：分开执行时，若 INCR 后进程崩溃，计数 key 永不过期，该用户被永久限流。
- **热点 key**：单个活动的所有 key 在同一分片，Redis 单分片可承受的 QPS 有限【R-20】。前面的限流与本地售罄标记已过滤绝大部分请求；如仍不够，可把库存拆成 N 个子 key（`sk:{aid#1}:stock` …）分散到不同分片，随机选一个扣减，代价是售罄判断与一人一单需要跨分片处理（见第 5 节）。

4.4.8 对账

每分钟对进行中的活动执行：

- Redis：`GET stock`、`SCARD users`（Pipeline 一次往返）【R-27】；
- MySQL：`SELECT COUNT(*) FROM seckill_order WHERE activity_id = ? AND status <> 2`（走 `idx_activity_status`）【M-26】；
- 期望：`stock + buyers = total_stock`，且 `db_orders ⩽ buyers`（差值是在途消息）。不满足则计数并告警，由人工或补偿任务修复。对账查询可放到从库，注意主从延迟带来的短暂误报【M-28】【M-30】。

§ 5 Alternatives Considered（备选方案）

方案	做法	优点	不选的原因
A. 纯 MySQL 悲观锁	`SELECT ... FOR UPDATE` 后扣减并插入订单	实现简单、强一致	所有请求在热点行锁上排队，连接池耗尽，还可能出现死锁与锁等待超时【M-15】【M-17】【M-18】
B. 纯 MySQL 乐观锁	`UPDATE ... WHERE version = ?` 重试	不持锁等待	热点下冲突率接近 100%，大量无效重试，数据库 QPS 被打满【M-18】
C. Redis 分布式锁 + 普通命令	加锁 → GET → DECR → 释放锁	好理解	每次多 2 次往返，锁串行化后吞吐低；还要处理锁过期、误删、主从切换丢锁【R-23】【R-24】；Lua 一次完成，没有这些问题
D. Kafka / RocketMQ 代替 Stream	预扣成功后由应用发送消息	吞吐与堆积能力更强，RocketMQ 有事务消息、延迟消息	扣减和发消息不在一个原子操作里，进程在两步之间崩溃会产生「扣了库存没订单」，需要事务消息或本地消息表；还要多运维一套集群。流量更大或已有 MQ 平台时应改用（见第 6 节）
E. 同步下单	Lua 预扣成功后立即在请求内写 MySQL	用户立即拿到最终结果，无需轮询	MySQL 写入速率直接暴露给峰值；请求耗时取决于数据库
F. 库存分片	把库存拆到 N 个 key/分片	突破单分片 QPS	一人一单需跨分片判重，售罄判断要汇总；当前规模用不到，作为扩展点
G. MySQL 内合并扣减	把多个扣减请求在应用层合并成一次 `stock = stock - n`	降低热点行锁次数	实现复杂，需要处理部分成功；本方案已经用队列把写入串行化

§ 6 Cross-cutting Concerns（横切关注点）

6.1 安全

- **身份**：只信任网关注入的 `X-User-ID`；服务不应直接暴露公网。示例直接读头，生产应校验网关签名或 mTLS。
- **防脚本**：一次性令牌（`crypto/rand` 16 字节，60 s，`GETDEL` 只能用一次），开售前拿不到令牌；配合网关验证码、风控黑名单。
- **注入**：所有 SQL 使用 `?` 占位符参数化，杜绝 SQL 注入；Lua 参数通过 `KEYS/ARGV` 传入，不拼接脚本。

- **管理接口**：`X-Admin-Token` 用 `subtle.ConstantTimeCompare` 比较，防计时攻击；生产应接入统一权限系统并只在内网开放。
- **输入**：请求体用 `http.MaxBytesReader` 限制 1 KB；路径 ID 校验为正整数；内部错误只返回 `internal error`，细节写日志。

6.2 可靠性

故障	影响	应对
Redis 主节点宕机	哨兵/Cluster 切换期间预扣不可用；异步复制可能丢失最近的扣减与消息【R-13】【R-14】【R-16】	切换期间请求失败（返回 5xx，用户重试）；丢失的扣减由 MySQL 条件更新与唯一索引兜底，**不会超卖**；丢失的消息会造成少卖，由对账发现
Redis 重启	依赖持久化恢复：AOF `everysec` 最多丢约 1 s；推荐开启 RDB+AOF 混合持久化，加快恢复【R-8】【R-9】【R-10】	重新预热时 `SETNX` 不覆盖已有库存；若库存 key 丢失，需人工按「total − DB 订单数」重建，不可直接用初始库存
内存不足被淘汰	秒杀 key 被淘汰会导致数据错乱	秒杀 Redis 独立部署，`maxmemory-policy` 设为 `noeviction` 或 `volatile-*` 并控制其他 key【R-11】【R-12】
MySQL 不可用 / 慢	消费失败	消息留在 PEL，自动重试；超过次数进死信并回滚预扣；用户看到「排队中」直至超时
消费者进程崩溃	已读未 ACK 的消息	其他实例 `XCLAIM` 接管；消费幂等（订单号 + 唯一索引）保证不重复下单
雪花 ID 时钟回拨	ID 重复	回拨 ≤ 5 ms 等待追平，否则报错拒绝；`NODE_ID` 由部署系统保证唯一
实例下线	处理中的请求	`signal.NotifyContext` 收到 SIGTERM 后 `Server.Shutdown` 优雅退出

- 重要取舍：**宁可少卖，不可超卖**。所有不确定情况（Redis 丢数据、MySQL 已售罄）都倾向于拒绝订单，而非放行。

6.3 可观测性

- **指标**（expvar，`GET /debug/vars`，生产可换 Prometheus）：请求数、限流数、售罄数、重复数、入队数、下单成功/失败数、死信数、对账不一致数。
- **需额外监控**：Stream 积压长度（`XLEN` / `XPENDING` 汇总）、Redis 慢查询与内存【R-22】、MySQL 慢 SQL 与行锁等待【M-24】、连接池使用率、主从延迟【M-30】。
- **日志**：`log/slog` JSON 结构化日志，关键字段 `activity`、`uid`、`order_id`、消息 ID，可串联「预扣 → 消息 → 落库」全链路。
- **告警**：死信数 > 0、对账不一致、积压持续增长、429 比例异常。

6.4 性能与容量（估算方法）

- MySQL 写入次数 ≈ 库存量（每件成功商品一次事务），与请求量无关；
- Redis 每个通过限流的请求约 4 次往返（用户限流计数、活动详情缓存、令牌 `GETDEL`、Lua 预扣）；
- 具体容量需按目标 QPS 压测得出，本文不给未经验证的数字。

§ 7 Degree of Constraint（约束程度）

- **强约束**：语言与基础设施固定为 Go + MySQL 8.0 + Redis 6.2+（`GETDEL` 需要 6.2），不引入新的中间件；不超卖、一人一单是硬性业务约束。
- **弱约束**：在这些约束内属于全新服务（greenfield），API、表结构、key 设计可以自由决定；部署形态（API 与消费者同进程或拆分）可以按流量调整。
- **随规模变化的决策**：若库存或流量增长到单分片 Redis 无法承载，或公司已有成熟 MQ 平台，应重新评估方案 D（外部 MQ + 事务消息）与方案 F（库存分片）。

§8 设计点 → 知识点对照

设计点	关联知识点
Lua 原子预扣、脚本要短	【R-1】单线程模型、【R-27】Lua/事务/Pipeline、【R-22】变慢排查、【R-29】秒杀与防超卖
Set 判重、Stream 队列、key 类型选择	【R-3】数据类型、【R-20】大 key / 热 key
Hash Tag 同槽	【R-15】Cluster 哈希槽
活动缓存穿透/击穿/雪崩	【R-17】【R-18】【R-19】
先更新库再删缓存	【R-25】Cache Aside、【R-26】其他读写策略
限流	【R-28】限流、【R-27】Lua
不用分布式锁	【R-23】【R-24】分布式锁
Redis 故障与少卖	【R-8】~【R-14】持久化/淘汰/复制/哨兵、【R-16】脑裂
条件更新防超卖	【M-14】当前读、【M-15】【M-16】锁与加锁规则、【M-13】MVCC
热点行事务顺序	【M-15】行锁、【M-17】缩短事务/死锁
唯一索引一人一单	【M-9】唯一索引、【M-37】索引分类、【M-38】避免重复插入
订单事务与持久性	【M-10】ACID、【M-19】redo、【M-20】undo（回滚）、【M-21】binlog、【M-22】两阶段提交
表与索引设计	【M-2】InnoDB、【M-4】【M-5】B+ 树/聚簇索引、【M-7】最左前缀、【M-34】DECIMAL、【M-36】反范式
雪花 ID	【M-31】分库分表与全局 ID
乐观锁编辑活动	【M-18】乐观/悲观锁
对账	【M-26】COUNT、【M-28】【M-30】主从复制与延迟

二、代码实现

> 代码与仓库根目录 `seckill/` 完全一致（https://github.com/LosAveRoad/interview-prep/tree/main/seckill）。依赖：`github.com/redis/go-redis/v9`、`github.com/go-sql-driver/mysql`（database/sql 驱动）、`golang.org/x/time/rate`、`golang.org/x/sync/singleflight`。只做编译与静态检查，未连接真实数据库运行。

项目结构与使用

```
./cmd/seckill/main.go
./go.mod
./go.sum
./internal/api/handler.go
./internal/cache/cache.go
./internal/cache/keys.go
./internal/cache/lua/deduct.lua
./internal/cache/lua/ratelimit.lua
```

```
./internal/cache/lua/rollback.lua
./internal/config/config.go
./internal/idgen/snowflake.go
./internal/metrics/metrics.go
./internal/model/model.go
./internal/mq/stream.go
./internal/service/seckill.go
./internal/store/mysql.go
./sql/schema.sql
```

- **编译检查**：`cd seckill && go build ./... && go vet ./...`
- **初始化数据库**：`mysql < sql/schema.sql`
- **运行**（需本地 MySQL 与 Redis 6.2+）：`MYSQL_DSN=... REDIS_ADDR=... NODE_ID=1 go run ./cmd/seckill`
- **调用顺序**：预热 `POST /admin/activities/1/preheat` → 获取令牌 `POST /api/v1/activities/1/token` → 秒杀 `POST /api/v1/activities/1/seckill` → 轮询 `GET /api/v1/activities/1/result`（后三者带 `X-User-ID` 头）。

sql/schema.sql

MySQL DDL：两张表与索引（4.3）。

```sql
-- 秒杀系统 MySQL DDL（MySQL 8.0, InnoDB, utf8mb4）
CREATE DATABASE IF NOT EXISTS seckill DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE seckill;

-- 秒杀活动表：一行 = 一个活动（一个商品的一场秒杀）
CREATE TABLE IF NOT EXISTS seckill_activity (
    id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '活动 ID，自增主键，顺序写入避免页分裂',
    goods_id    BIGINT UNSIGNED NOT NULL COMMENT '商品 ID',
    title       VARCHAR(128)    NOT NULL DEFAULT '' COMMENT '活动标题',
    price       DECIMAL(10, 2)  NOT NULL COMMENT '秒杀价，金额用 DECIMAL',
    total_stock INT UNSIGNED    NOT NULL COMMENT '初始库存',
    stock       INT UNSIGNED    NOT NULL COMMENT '数据库剩余库存（最终兜底）',
    start_time  DATETIME        NOT NULL COMMENT '开始时间',
    end_time    DATETIME        NOT NULL COMMENT '结束时间',
    status      TINYINT         NOT NULL DEFAULT 0 COMMENT '0 草稿 1 已发布 2 已下线',
    version     INT UNSIGNED    NOT NULL DEFAULT 0 COMMENT '乐观锁版本号（后台编辑用）',
    created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_status_start (status, start_time) COMMENT '预热任务按状态 + 开始时间扫描'
) ENGINE = InnoDB COMMENT = '秒杀活动';

-- 秒杀订单表：一人一单由唯一索引兜底
CREATE TABLE IF NOT EXISTS seckill_order (
    id          BIGINT UNSIGNED NOT NULL COMMENT '订单 ID，雪花算法生成，趋势递增',
    activity_id BIGINT UNSIGNED NOT NULL COMMENT '活动 ID',
    user_id     BIGINT UNSIGNED NOT NULL COMMENT '用户 ID',
    goods_id    BIGINT UNSIGNED NOT NULL COMMENT '商品 ID（冗余，减少 JOIN）',
    price       DECIMAL(10, 2)  NOT NULL COMMENT '成交价',
    status      TINYINT         NOT NULL DEFAULT 0 COMMENT '0 待支付 1 已支付 2 已取消',
    created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_user_activity (user_id, activity_id) COMMENT '一人一单；user_id 在前，兼顾「我的订单」查询',
    KEY idx_activity_status (activity_id, status) COMMENT '对账、按活动统计'
) ENGINE = InnoDB COMMENT = '秒杀订单';
```

internal/cache/lua/deduct.lua

库存预扣 + 一人一单 + 入队，原子执行（4.4.2）。

```lua
-- 库存预扣 + 一人一单 + 投递订单消息，整段脚本在 Redis 中原子执行。
-- 所有 KEY 使用同一个 hash tag {activityID}，保证 Redis Cluster 下落在同一个 slot。
-- KEYS[1] 库存        sk:{aid}:stock   (string, 整数)
-- KEYS[2] 已购用户    sk:{aid}:users   (set)
-- KEYS[3] 订单消息流  sk:{aid}:orders  (stream)
-- KEYS[4] 抢购结果    sk:{aid}:result:{uid} (string)
-- ARGV[1] user_id  ARGV[2] order_id  ARGV[3] 请求时间戳(ms)  ARGV[4] 结果 TTL(秒)
-- 返回: >=0 扣减后剩余库存; -1 已售罄; -2 重复购买; -3 活动未预热
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
    return -2
end
local stock = redis.call('GET', KEYS[1])
if not stock then
    return -3
end
stock = tonumber(stock)
if stock <= 0 then
    return -1
end
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
redis.call('XADD', KEYS[3], 'MAXLEN', '~', '1000000', '*',
    'uid', ARGV[1], 'oid', ARGV[2], 'ts', ARGV[3])
redis.call('SET', KEYS[4], 'queued:' .. ARGV[2], 'EX', ARGV[4])
return stock - 1
```

internal/cache/lua/rollback.lua

幂等回滚预扣（4.4.5）。

```lua
-- 下单失败时回滚预扣: 幂等，只有用户仍在已购集合中才回滚。
-- KEYS[1] 库存  KEYS[2] 已购用户  KEYS[3] 抢购结果
-- ARGV[1] user_id  ARGV[2] 结果值  ARGV[3] 结果 TTL(秒)  ARGV[4] 是否归还库存(1/0)
if redis.call('SREM', KEYS[2], ARGV[1]) == 1 then
    if ARGV[4] == '1' then
        redis.call('INCR', KEYS[1])
    end
    redis.call('SET', KEYS[3], ARGV[2], 'EX', ARGV[3])
    return 1
end
return 0
```

internal/cache/lua/ratelimit.lua

用户级固定窗口限流（4.4.7）。

```lua
-- 固定窗口限流: INCR 与 EXPIRE 在同一脚本内执行，避免产生没有过期时间的计数 key。
-- KEYS[1] 计数 key  ARGV[1] 窗口内最大请求数  ARGV[2] 窗口长度(秒)
-- 返回 1 放行, 0 拒绝
local n = redis.call('INCR', KEYS[1])
if n == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[2])
end
if n > tonumber(ARGV[1]) then
    return 0
end
return 1
```

internal/cache/keys.go

Redis key 与 Hash Tag（4.3）。

```go
package cache

import "fmt"

// Redis key 设计：业务前缀 sk + hash tag {activityID}。
// 同一活动的库存、已购集合、消息流、结果放在同一个 slot，Lua 脚本才能在 Cluster 模式下执行。

// StockKey 预扣库存（String）。
func StockKey(aid int64) string { return fmt.Sprintf("sk:{%d}:stock", aid) }

// UsersKey 已抢到的用户集合（Set），实现一人一单。
func UsersKey(aid int64) string { return fmt.Sprintf("sk:{%d}:users", aid) }

// StreamKey 订单消息流（Stream），每个活动一个，避免跨 slot。
func StreamKey(aid int64) string { return fmt.Sprintf("sk:{%d}:orders", aid) }

// InfoKey 活动详情缓存（String/JSON），Cache Aside。
func InfoKey(aid int64) string { return fmt.Sprintf("sk:{%d}:info", aid) }

// ResultKey 用户抢购结果（String），供前端轮询。
func ResultKey(aid, uid int64) string { return fmt.Sprintf("sk:{%d}:result:%d", aid, uid) }

// TokenKey 秒杀令牌（String），隐藏真实秒杀接口、一次性使用。
func TokenKey(aid, uid int64) string { return fmt.Sprintf("sk:{%d}:token:%d", aid, uid) }

// UserRateKey 用户级限流计数（String），按秒分窗口。
func UserRateKey(uid, sec int64) string { return fmt.Sprintf("sk:rl:%d:%d", uid, sec) }

// DeadLetterKey 死信流，记录多次重试仍失败的下单消息。
const DeadLetterKey = "sk:dlq"
```

internal/cache/cache.go

go-redis 封装：预热、预扣、回滚、令牌、Cache Aside（4.4.1 / 4.4.2 / 4.4.6）。

```go
// Package cache 封装秒杀相关的 Redis 操作。
package cache

import (
    "context"
    _ "embed"
    "encoding/json"
    "errors"
    "math/rand/v2"
    "strconv"
    "strings"
    "time"

    "github.com/redis/go-redis/v9"
    "golang.org/x/sync/singleflight"

    "github.com/lisa/seckill/internal/model"
)

var (
    //go:embed lua/deduct.lua
    deductSrc string
    //go:embed lua/rollback.lua
    rollbackSrc string
    //go:embed lua/ratelimit.lua
    rateLimitSrc string
```

```
    // redis.Script 先用 EVALSHA 执行，脚本未缓存（NOSCRIPT）时自动回退到 EVAL。
    deductScript    = redis.NewScript(deductSrc)
    rollbackScript  = redis.NewScript(rollbackSrc)
    rateLimitScript = redis.NewScript(rateLimitSrc)
)

// Deduct 的返回码。
const (
    DeductSoldOut      int64 = -1
    DeductRepeat       int64 = -2
    DeductNotPreheated int64 = -3
)

// emptyPlaceholder 缓存空值，防止缓存穿透。
const emptyPlaceholder = "<nil>"

// ErrActivityNotFound 表示活动不存在（含命中空值缓存）。
var ErrActivityNotFound = errors.New("cache: activity not found")

// Cache 是 Redis 访问层。
type Cache struct {
    rdb       redis.UniversalClient
    sf        singleflight.Group
    resultTTL time.Duration
}

// New 创建 Cache。UniversalClient 兼容单机、哨兵与 Cluster 模式。
func New(rdb redis.UniversalClient, resultTTL time.Duration) *Cache {
    return &Cache{rdb: rdb, resultTTL: resultTTL}
}

// Client 返回底层客户端（供消息队列使用）。
func (c *Cache) Client() redis.UniversalClient { return c.rdb }

// Preheat 把活动库存加载到 Redis。使用 SETNX：活动进行中重复预热不会覆盖已扣减的库存。
func (c *Cache) Preheat(ctx context.Context, a *model.Activity) (bool, error) {
    ttl := time.Until(a.EndTime) + 24*time.Hour // 活动结束后再保留一天用于对账
    return c.rdb.SetNX(ctx, StockKey(a.ID), a.Stock, ttl).Result()
}

// Deduct 执行库存预扣 Lua 脚本，返回剩余库存或负数返回码。
func (c *Cache) Deduct(ctx context.Context, aid, uid, oid int64) (int64, error) {
    keys := []string{StockKey(aid), UsersKey(aid), StreamKey(aid), ResultKey(aid, uid)}
    return deductScript.Run(ctx, c.rdb, keys,
        uid, oid, time.Now().UnixMilli(), int64(c.resultTTL/time.Second)).Int64()
}

// Rollback 回滚预扣（幂等）。restoreStock=false 用于数据库已售罄、不应再归还库存的场景。
func (c *Cache) Rollback(ctx context.Context, aid, uid int64, result string, restoreStock bool) (bool, error) {
    restore := "0"
    if restoreStock {
        restore = "1"
    }
    keys := []string{StockKey(aid), UsersKey(aid), ResultKey(aid, uid)}
    n, err := rollbackScript.Run(ctx, c.rdb, keys,
        uid, result, int64(c.resultTTL/time.Second), restore).Int64()
    return n == 1, err
}

// AllowUser 用户级限流：每秒最多 limit 次请求。
func (c *Cache) AllowUser(ctx context.Context, uid int64, limit int) (bool, error) {
    key := UserRateKey(uid, time.Now().Unix())
    n, err := rateLimitScript.Run(ctx, c.rdb, []string{key}, limit, 2).Int64()
    return n == 1, err
}

// SetToken 保存一次性秒杀令牌。
func (c *Cache) SetToken(ctx context.Context, aid, uid int64, token string, ttl time.Duration) error {
```

```
    return c.rdb.Set(ctx, TokenKey(aid, uid), token, ttl).Err()
}

// ConsumeToken 原子地取出并删除令牌（GETDEL，Redis 6.2+），保证令牌只能使用一次。
func (c *Cache) ConsumeToken(ctx context.Context, aid, uid int64, token string) (bool, error) {
    v, err := c.rdb.GetDel(ctx, TokenKey(aid, uid)).Result()
    if errors.Is(err, redis.Nil) {
        return false, nil
    }
    if err != nil {
        return false, err
    }
    return v == token, nil
}

// SetResult 写入抢购结果。
func (c *Cache) SetResult(ctx context.Context, aid, uid int64, result string) error {
    return c.rdb.Set(ctx, ResultKey(aid, uid), result, c.resultTTL).Err()
}

// GetResult 读取抢购结果，不存在返回空串。
func (c *Cache) GetResult(ctx context.Context, aid, uid int64) (string, error) {
    v, err := c.rdb.Get(ctx, ResultKey(aid, uid)).Result()
    if errors.Is(err, redis.Nil) {
        return "", nil
    }
    return v, err
}

// GetActivity 按 Cache Aside 读取活动详情：
//   - 命中直接返回；命中空值占位符返回 ErrActivityNotFound（防穿透）；
//   - 未命中时用 singleflight 合并并发回源（防击穿）；
//   - 回写缓存时 TTL 加随机抖动（防雪崩）。
func (c *Cache) GetActivity(ctx context.Context, aid int64,
    load func(context.Context, int64) (*model.Activity, error)) (*model.Activity, error) {

    key := InfoKey(aid)
    if raw, err := c.rdb.Get(ctx, key).Result(); err == nil {
        if raw == emptyPlaceholder {
            return nil, ErrActivityNotFound
        }
        var a model.Activity
        if err := json.Unmarshal([]byte(raw), &a); err == nil {
            return &a, nil
        }
    } else if !errors.Is(err, redis.Nil) {
        // Redis 故障时降级直接查库（由上层限流保护数据库）
        return load(ctx, aid)
    }

    v, err, _ := c.sf.Do(key, func() (any, error) {
        a, err := load(ctx, aid)
        if errors.Is(err, ErrActivityNotFound) {
            _ = c.rdb.Set(ctx, key, emptyPlaceholder, time.Minute).Err()
            return nil, err
        }
        if err != nil {
            return nil, err
        }
        if b, err := json.Marshal(a); err == nil {
            ttl := 10*time.Minute + time.Duration(rand.IntN(120))*time.Second
            _ = c.rdb.Set(ctx, key, b, ttl).Err()
        }
        return a, nil
    })
    if err != nil {
        return nil, err
    }
    return v.(*model.Activity), nil
```

```
}

// InvalidateActivity 删除活动缓存（先更新数据库，再删除缓存）。
func (c *Cache) InvalidateActivity(ctx context.Context, aid int64) error {
    return c.rdb.Del(ctx, InfoKey(aid)).Err()
}

// StockSnapshot 读取 Redis 中的剩余库存与已购人数，用于对账。
func (c *Cache) StockSnapshot(ctx context.Context, aid int64) (stock, buyers int64, err error) {
    pipe := c.rdb.Pipeline()
    stockCmd := pipe.Get(ctx, StockKey(aid))
    buyersCmd := pipe.SCard(ctx, UsersKey(aid))
    if _, err = pipe.Exec(ctx); err != nil && !errors.Is(err, redis.Nil) {
        return 0, 0, err
    }
    stock, _ = strconv.ParseInt(stockCmd.Val(), 10, 64)
    return stock, buyersCmd.Val(), nil
}

// ParseResult 解析结果字符串 "queued:<oid>" / "success:<oid>" / "failed:<reason>"。
func ParseResult(v string) (status, detail string) {
    status, detail, _ = strings.Cut(v, ":")
    return status, detail
}
```

internal/store/mysql.go

database/sql：订单事务、条件扣减、唯一键冲突识别（4.4.3 / 4.4.4 / 4.4.5）。

```
// Package store 是 MySQL 访问层（database/sql + go-sql-driver/mysql）。
package store

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "time"

    "github.com/go-sql-driver/mysql"

    "github.com/lisa/seckill/internal/model"
)

var (
    // ErrNotFound 记录不存在。
    ErrNotFound = errors.New("store: not found")
    // ErrDuplicateOrder 违反唯一索引 uk_user_activity，即同一用户重复下单。
    ErrDuplicateOrder = errors.New("store: duplicate order")
    // ErrSoldOut 数据库库存已为 0（条件更新影响 0 行）。
    ErrSoldOut = errors.New("store: sold out")
    // ErrVersionConflict 乐观锁版本冲突。
    ErrVersionConflict = errors.New("store: version conflict")
)

// mysqlErrDupEntry 是 MySQL 唯一键冲突错误码。
const mysqlErrDupEntry = 1062

// Store 封装数据库操作。
type Store struct{ db *sql.DB }

// Open 打开连接池并设置池参数。
func Open(dsn string) (*Store, error) {
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        return nil, err
```

```
    }
    db.SetMaxOpenConns(100) // 上限要小于 MySQL max_connections / 实例数
    db.SetMaxIdleConns(50)
    db.SetConnMaxLifetime(30 * time.Minute) // 小于服务端 wait_timeout
    db.SetConnMaxIdleTime(5 * time.Minute)
    return &Store{db: db}, nil
}

// Ping 检查连通性。
func (s *Store) Ping(ctx context.Context) error { return s.db.PingContext(ctx) }

// Close 关闭连接池。
func (s *Store) Close() error { return s.db.Close() }

const activityCols = `id, goods_id, title, price, total_stock, stock, start_time, end_time, status, version`

func scanActivity(row interface{ Scan(...any) error }) (*model.Activity, error) {
    var a model.Activity
    err := row.Scan(&a.ID, &a.GoodsID, &a.Title, &a.Price, &a.TotalStock, &a.Stock,
        &a.StartTime, &a.EndTime, &a.Status, &a.Version)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    return &a, err
}

// GetActivity 按主键查询活动（聚簇索引，一次 B+ 树查找）。
func (s *Store) GetActivity(ctx context.Context, id int64) (*model.Activity, error) {
    row := s.db.QueryRowContext(ctx,
        `SELECT `+activityCols+` FROM seckill_activity WHERE id = ?`, id)
    return scanActivity(row)
}

// ListUpcoming 查询即将开始或进行中的已发布活动，命中索引 idx_status_start。
func (s *Store) ListUpcoming(ctx context.Context, before time.Time) ([]*model.Activity, error) {
    rows, err := s.db.QueryContext(ctx,
        `SELECT `+activityCols+` FROM seckill_activity
         WHERE status = ? AND start_time <= ? AND end_time > NOW()`,
        model.ActivityPublished, before)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var list []*model.Activity
    for rows.Next() {
        a, err := scanActivity(rows)
        if err != nil {
            return nil, err
        }
        list = append(list, a)
    }
    return list, rows.Err()
}

// UpdateActivityInfo 后台修改活动标题/价格，使用版本号乐观锁。
func (s *Store) UpdateActivityInfo(ctx context.Context, a *model.Activity) error {
    res, err := s.db.ExecContext(ctx,
        `UPDATE seckill_activity SET title = ?, price = ?, version = version + 1
         WHERE id = ? AND version = ?`, a.Title, a.Price, a.ID, a.Version)
    if err != nil {
        return err
    }
    if n, _ := res.RowsAffected(); n == 0 {
        return ErrVersionConflict
    }
    return nil
}

// CreateOrder 在一个事务内创建订单并扣减数据库库存。
```

```go
//
// 1. 先 INSERT 订单：唯一索引 uk_user_activity 保证一人一单，冲突立即返回，不碰热点行；
// 2. 后 UPDATE 库存：WHERE stock > 0 的条件更新是防超卖的最后一道防线；
//    UPDATE 会对活动行加 X 型记录锁，放在事务最后执行，持锁时间最短。
func (s *Store) CreateOrder(ctx context.Context, o *model.Order) (err error) {
    tx, err := s.db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer func() {
        if err != nil {
            _ = tx.Rollback()
        }
    }()

    if _, err = tx.ExecContext(ctx,
        `INSERT INTO seckill_order (id, activity_id, user_id, goods_id, price, status)
         VALUES (?, ?, ?, ?, ?, ?)`,
        o.ID, o.ActivityID, o.UserID, o.GoodsID, o.Price, model.OrderUnpaid); err != nil {
        var me *mysql.MySQLError
        if errors.As(err, &me) && me.Number == mysqlErrDupEntry {
            return ErrDuplicateOrder
        }
        return fmt.Errorf("insert order: %w", err)
    }

    res, err := tx.ExecContext(ctx,
        `UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`, o.ActivityID)
    if err != nil {
        return fmt.Errorf("deduct stock: %w", err)
    }
    n, err := res.RowsAffected()
    if err != nil {
        return err
    }
    if n == 0 {
        err = ErrSoldOut
        return err
    }
    return tx.Commit()
}

// GetOrderByUser 通过唯一索引查询用户在某活动下的订单。
func (s *Store) GetOrderByUser(ctx context.Context, aid, uid int64) (*model.Order, error) {
    var o model.Order
    err := s.db.QueryRowContext(ctx,
        `SELECT id, activity_id, user_id, goods_id, price, status, created_at
         FROM seckill_order WHERE user_id = ? AND activity_id = ?`, uid, aid).
        Scan(&o.ID, &o.ActivityID, &o.UserID, &o.GoodsID, &o.Price, &o.Status, &o.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    return &o, err
}

// CountOrders 统计活动的有效订单数（对账用，走 idx_activity_status 覆盖索引）。
func (s *Store) CountOrders(ctx context.Context, aid int64) (int64, error) {
    var n int64
    err := s.db.QueryRowContext(ctx,
        `SELECT COUNT(*) FROM seckill_order WHERE activity_id = ? AND status <> ?`,
        aid, model.OrderCanceled).Scan(&n)
    return n, err
}
```

internal/mq/stream.go

Redis Stream 消费者组：ACK、PEL 重试、死信（4.4.5）。

```go
// Package mq 基于 Redis Stream 实现异步下单消息队列（消费者组 + ACK + 重试 + 死信）。
package mq

import (
    "context"
    "errors"
    "log/slog"
    "strconv"
    "strings"
    "time"

    "github.com/redis/go-redis/v9"
)

// OrderMsg 是下单消息。
type OrderMsg struct {
    ActivityID int64
    UserID     int64
    OrderID    int64
    TS         int64 // 用户请求时间（毫秒）
}

// ErrPermanent 包装不可重试的错误（如消息格式错误），消费者直接 ACK。
var ErrPermanent = errors.New("mq: permanent error")

// Handler 处理一条消息。返回 nil 或 ErrPermanent 时 ACK；其他错误保留在 PEL 中等待重试。
type Handler func(ctx context.Context, m OrderMsg) error

// DeadLetterFunc 在消息超过最大投递次数后调用，用于补偿（回滚预扣）。
type DeadLetterFunc func(ctx context.Context, m OrderMsg)

// EnsureGroup 创建消费者组（流不存在时一并创建），已存在则忽略。
func EnsureGroup(ctx context.Context, rdb redis.UniversalClient, stream, group string) error {
    err := rdb.XGroupCreateMkStream(ctx, stream, group, "0").Err()
    if err != nil && !strings.HasPrefix(err.Error(), "BUSYGROUP") {
        return err
    }
    return nil
}

// Consumer 消费一个活动的订单流。
type Consumer struct {
    RDB        redis.UniversalClient
    Stream     string
    Group      string
    Name       string
    ActivityID int64
    MaxRetry   int64
    ClaimIdle  time.Duration
    DLQ        string
    Handle     Handler
    OnDead     DeadLetterFunc
    Log        *slog.Logger
}

// Run 阻塞运行直到 ctx 取消。
func (c *Consumer) Run(ctx context.Context) {
    go c.reclaimLoop(ctx)
    for ctx.Err() == nil {
        streams, err := c.RDB.XReadGroup(ctx, &redis.XReadGroupArgs{
            Group:    c.Group,
            Consumer: c.Name,
            Streams:  []string{c.Stream, ">"}, // ">" 只读取从未投递过的新消息
            Count:    32,
            Block:    2 * time.Second,
        }).Result()
        if errors.Is(err, redis.Nil) || ctx.Err() != nil {
            continue
```

```
            }
            if err != nil {
                c.Log.Error("xreadgroup", "stream", c.Stream, "err", err)
                time.Sleep(time.Second)
                continue
            }
            for _, s := range streams {
                for _, msg := range s.Messages {
                    c.process(ctx, msg)
                }
            }
        }
    }

    func (c *Consumer) process(ctx context.Context, msg redis.XMessage) {
        m, err := c.decode(msg)
        if err == nil {
            err = c.Handle(ctx, m)
        }
        if err != nil && !errors.Is(err, ErrPermanent) {
            // 不 ACK：消息留在 PEL（Pending Entries List），由 reclaimLoop 重新认领
            c.Log.Warn("handle order failed, will retry", "id", msg.ID, "err", err)
            return
        }
        if err != nil {
            c.Log.Error("drop bad message", "id", msg.ID, "err", err)
        }
        if err := c.RDB.XAck(ctx, c.Stream, c.Group, msg.ID).Err(); err != nil {
            c.Log.Error("xack", "id", msg.ID, "err", err)
        }
    }

    // reclaimLoop 周期性检查 PEL：空闲过久的消息重新认领处理；超过最大投递次数的进入死信。
    func (c *Consumer) reclaimLoop(ctx context.Context) {
        t := time.NewTicker(c.ClaimIdle / 2)
        defer t.Stop()
        for {
            select {
            case <-ctx.Done():
                return
            case <-t.C:
            }
            pending, err := c.RDB.XPendingExt(ctx, &redis.XPendingExtArgs{
                Stream: c.Stream, Group: c.Group, Idle: c.ClaimIdle,
                Start: "-", End: "+", Count: 100,
            }).Result()
            if err != nil {
                c.Log.Error("xpending", "err", err)
                continue
            }
            for _, p := range pending {
                msgs, err := c.RDB.XClaim(ctx, &redis.XClaimArgs{
                    Stream: c.Stream, Group: c.Group, Consumer: c.Name,
                    MinIdle: c.ClaimIdle, Messages: []string{p.ID},
                }).Result()
                if err != nil || len(msgs) == 0 {
                    continue // 已被其他消费者认领
                }
                if p.RetryCount >= c.MaxRetry {
                    c.deadLetter(ctx, msgs[0])
                    continue
                }
                c.process(ctx, msgs[0])
            }
        }
    }

    func (c *Consumer) deadLetter(ctx context.Context, msg redis.XMessage) {
        values := map[string]any{"stream": c.Stream, "msg_id": msg.ID}
```

```
    for k, v := range msg.Values {
        values[k] = v
    }
    if err := c.RDB.XAdd(ctx, &redis.XAddArgs{Stream: c.DLQ, Values: values}).Err(); err != nil {
        c.Log.Error("write dlq", "id", msg.ID, "err", err)
        return // 写死信失败则不 ACK，下轮再试
    }
    if m, err := c.decode(msg); err == nil && c.OnDead != nil {
        c.OnDead(ctx, m)
    }
    _ = c.RDB.XAck(ctx, c.Stream, c.Group, msg.ID).Err()
    c.Log.Error("message moved to dead letter queue", "id", msg.ID)
}

func (c *Consumer) decode(msg redis.XMessage) (OrderMsg, error) {
    get := func(k string) (int64, error) {
        s, _ := msg.Values[k].(string)
        n, err := strconv.ParseInt(s, 10, 64)
        if err != nil {
            return 0, errors.Join(ErrPermanent, err)
        }
        return n, nil
    }
    uid, err1 := get("uid")
    oid, err2 := get("oid")
    ts, _ := get("ts")
    if err := errors.Join(err1, err2); err != nil {
        return OrderMsg{}, err
    }
    return OrderMsg{ActivityID: c.ActivityID, UserID: uid, OrderID: oid, TS: ts}, nil
}
```

internal/service/seckill.go

业务编排：秒杀主流程、幂等消费、预热、对账（4.4）。

```
// Package service 实现秒杀核心业务流程。
package service

import (
    "context"
    "crypto/rand"
    "encoding/hex"
    "errors"
    "fmt"
    "log/slog"
    "strconv"
    "sync"
    "time"

    "github.com/lisa/seckill/internal/cache"
    "github.com/lisa/seckill/internal/config"
    "github.com/lisa/seckill/internal/idgen"
    "github.com/lisa/seckill/internal/metrics"
    "github.com/lisa/seckill/internal/model"
    "github.com/lisa/seckill/internal/mq"
    "github.com/lisa/seckill/internal/store"
)

// 业务错误，由 API 层映射为 HTTP 状态码。
var (
    ErrNotFound     = errors.New("activity not found")
    ErrNotStarted   = errors.New("activity not started")
    ErrEnded        = errors.New("activity ended")
    ErrSoldOut      = errors.New("sold out")
    ErrRepeat       = errors.New("already purchased")
```

```
    ErrNotPreheated = errors.New("activity not preheated")
    ErrBadToken     = errors.New("invalid seckill token")
    ErrTooMany      = errors.New("too many requests")
)

// Result 是抢购结果。
type Result struct {
    Status  string `json:"status"` // queued / success / failed / none
    OrderID string `json:"order_id,omitempty"`
    Reason  string `json:"reason,omitempty"`
}

// Service 串联 Redis、MySQL 与消息队列。
type Service struct {
    cfg   config.Config
    cache *cache.Cache
    store *store.Store
    ids   *idgen.Snowflake
    log   *slog.Logger

    soldOut   sync.Map // 本地售罄标记：activityID -> struct{}，售罄后请求不再访问 Redis
    consumers sync.Map // 已启动消费者的活动：activityID -> struct{}
    bgCtx     context.Context
}

// New 创建 Service。bgCtx 控制后台消费者的生命周期。
func New(bgCtx context.Context, cfg config.Config, c *cache.Cache, s *store.Store,
    ids *idgen.Snowflake, log *slog.Logger) *Service {
    return &Service{cfg: cfg, cache: c, store: s, ids: ids, log: log, bgCtx: bgCtx}
}

func (s *Service) loadActivity(ctx context.Context, id int64) (*model.Activity, error) {
    a, err := s.store.GetActivity(ctx, id)
    if errors.Is(err, store.ErrNotFound) {
        return nil, cache.ErrActivityNotFound
    }
    return a, err
}

// Activity 查询活动详情（Cache Aside）。
func (s *Service) Activity(ctx context.Context, id int64) (*model.Activity, error) {
    a, err := s.cache.GetActivity(ctx, id, s.loadActivity)
    if errors.Is(err, cache.ErrActivityNotFound) {
        return nil, ErrNotFound
    }
    return a, err
}

func (s *Service) checkTime(a *model.Activity) error {
    now := time.Now()
    switch {
    case a.Status != model.ActivityPublished:
        return ErrNotFound
    case now.Before(a.StartTime):
        return ErrNotStarted
    case !now.Before(a.EndTime):
        return ErrEnded
    }
    return nil
}

// IssueToken 活动开始后为用户签发一次性令牌（秒杀地址隐藏，防止脚本提前刷接口）。
func (s *Service) IssueToken(ctx context.Context, aid, uid int64) (string, error) {
    a, err := s.Activity(ctx, aid)
    if err != nil {
        return "", err
    }
    if err := s.checkTime(a); err != nil {
        return "", err
```

```
    }
    b := make([]byte, 16)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    token := hex.EncodeToString(b)
    return token, s.cache.SetToken(ctx, aid, uid, token, s.cfg.TokenTTL)
}

// Seckill 执行抢购：限流 → 校验 → 令牌 → 本地售罄标记 → Lua 预扣并投递消息。
// 成功只代表“排队中”，订单由消费者异步落库，返回预生成的订单号。
func (s *Service) Seckill(ctx context.Context, aid, uid int64, token string) (int64, error) {
    metrics.Requests.Add(1)

    if _, done := s.soldOut.Load(aid); done {
        metrics.SoldOut.Add(1)
        return 0, ErrSoldOut
    }
    ok, err := s.cache.AllowUser(ctx, uid, s.cfg.UserReqPerSec)
    if err != nil {
        return 0, err
    }
    if !ok {
        metrics.RateLimited.Add(1)
        return 0, ErrTooMany
    }
    a, err := s.Activity(ctx, aid)
    if err != nil {
        return 0, err
    }
    if err := s.checkTime(a); err != nil {
        return 0, err
    }
    valid, err := s.cache.ConsumeToken(ctx, aid, uid, token)
    if err != nil {
        return 0, err
    }
    if !valid {
        return 0, ErrBadToken
    }

    oid, err := s.ids.Next()
    if err != nil {
        return 0, err
    }
    left, err := s.cache.Deduct(ctx, aid, uid, oid)
    if err != nil {
        return 0, err
    }
    switch left {
    case cache.DeductSoldOut:
        s.soldOut.Store(aid, struct{}{})
        metrics.SoldOut.Add(1)
        return 0, ErrSoldOut
    case cache.DeductRepeat:
        metrics.Repeat.Add(1)
        return 0, ErrRepeat
    case cache.DeductNotPreheated:
        return 0, ErrNotPreheated
    }
    if left == 0 {
        s.soldOut.Store(aid, struct{}{})
    }
    metrics.Queued.Add(1)
    return oid, nil
}

// Result 查询抢购结果：先查 Redis，缺失时回查数据库。
func (s *Service) Result(ctx context.Context, aid, uid int64) (Result, error) {
```

```
    v, err := s.cache.GetResult(ctx, aid, uid)
    if err != nil {
        return Result{}, err
    }
    if v != "" {
        status, detail := cache.ParseResult(v)
        if status == "failed" {
            return Result{Status: status, Reason: detail}, nil
        }
        return Result{Status: status, OrderID: detail}, nil
    }
    o, err := s.store.GetOrderByUser(ctx, aid, uid)
    if errors.Is(err, store.ErrNotFound) {
        return Result{Status: "none"}, nil
    }
    if err != nil {
        return Result{}, err
    }
    return Result{Status: "success", OrderID: strconv.FormatInt(o.ID, 10)}, nil
}

// HandleOrder 是消息消费者：在 MySQL 事务中创建订单并扣减库存，保证幂等。
func (s *Service) HandleOrder(ctx context.Context, m mq.OrderMsg) error {
    a, err := s.Activity(ctx, m.ActivityID)
    if errors.Is(err, ErrNotFound) {
        return fmt.Errorf("%w: activity %d", mq.ErrPermanent, m.ActivityID)
    }
    if err != nil {
        return err
    }
    o := &model.Order{ID: m.OrderID, ActivityID: m.ActivityID, UserID: m.UserID,
        GoodsID: a.GoodsID, Price: a.Price}

    err = s.store.CreateOrder(ctx, o)
    switch {
    case err == nil:
        metrics.OrderCreated.Add(1)
        return s.cache.SetResult(ctx, m.ActivityID, m.UserID, "success:"+strconv.FormatInt(m.OrderID, 10))

    case errors.Is(err, store.ErrDuplicateOrder):
        // 消息重复投递（至少一次语义）或历史订单：订单号相同视为已成功，幂等返回
        exist, qerr := s.store.GetOrderByUser(ctx, m.ActivityID, m.UserID)
        if qerr != nil {
            return qerr
        }
        if exist.ID == m.OrderID {
            return s.cache.SetResult(ctx, m.ActivityID, m.UserID, "success:"+strconv.FormatInt(exist.ID, 10))
        }
        // 用户已有其他订单（例如 Redis 数据丢失后重复抢到），本次预扣作废并归还库存
        metrics.OrderFailed.Add(1)
        _, rerr := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:repeat", true)
        s.soldOut.Delete(m.ActivityID)
        return rerr

    case errors.Is(err, store.ErrSoldOut):
        // 数据库兜底发现已售罄（Redis 与 DB 不一致），失败且不归还 Redis 库存
        metrics.OrderFailed.Add(1)
        _, rerr := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:sold_out", false)
        return rerr
    }
    return err // 数据库暂时不可用等，返回错误交给 MQ 重试
}

// OnDeadLetter 消息多次重试仍失败：回滚预扣，让库存回到池子里。
func (s *Service) OnDeadLetter(ctx context.Context, m mq.OrderMsg) {
    metrics.DeadLetters.Add(1)
    if _, err := s.store.GetOrderByUser(ctx, m.ActivityID, m.UserID); err == nil {
        return // 订单其实已落库，无需回滚
    }
```

```
    if _, err := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:system_error", true); err != nil {
        s.log.Error("rollback dead letter", "aid", m.ActivityID, "uid", m.UserID, "err", err)
        return
    }
    s.soldOut.Delete(m.ActivityID)
}

// Preheat 预热即将开始的活动：库存写入 Redis、创建消费者组并启动该活动的消费者。
func (s *Service) Preheat(ctx context.Context, a *model.Activity) error {
    if _, err := s.cache.Preheat(ctx, a); err != nil {
        return err
    }
    stream := cache.StreamKey(a.ID)
    if err := mq.EnsureGroup(ctx, s.cache.Client(), stream, s.cfg.ConsumerGroup); err != nil {
        return err
    }
    if _, loaded := s.consumers.LoadOrStore(a.ID, struct{}{}); !loaded {
        c := &mq.Consumer{
            RDB: s.cache.Client(), Stream: stream, Group: s.cfg.ConsumerGroup,
            Name: s.cfg.ConsumerName, ActivityID: a.ID, MaxRetry: s.cfg.MaxRetry,
            ClaimIdle: s.cfg.ClaimIdle, DLQ: cache.DeadLetterKey,
            Handle: s.HandleOrder, OnDead: s.OnDeadLetter, Log: s.log,
        }
        go c.Run(s.bgCtx)
        s.log.Info("consumer started", "activity", a.ID)
    }
    return nil
}

// PreheatByID 管理接口：手动预热指定活动。
func (s *Service) PreheatByID(ctx context.Context, aid int64) error {
    a, err := s.store.GetActivity(ctx, aid)
    if errors.Is(err, store.ErrNotFound) {
        return ErrNotFound
    }
    if err != nil {
        return err
    }
    return s.Preheat(ctx, a)
}

// PreheatUpcoming 定时任务：预热未来 PreheatIn 内开始的活动。
func (s *Service) PreheatUpcoming(ctx context.Context) {
    list, err := s.store.ListUpcoming(ctx, time.Now().Add(s.cfg.PreheatIn))
    if err != nil {
        s.log.Error("list upcoming", "err", err)
        return
    }
    for _, a := range list {
        if err := s.Preheat(ctx, a); err != nil {
            s.log.Error("preheat", "activity", a.ID, "err", err)
        }
    }
}

// UpdateActivityInfo 后台修改活动：先更新数据库，再删除缓存。
func (s *Service) UpdateActivityInfo(ctx context.Context, a *model.Activity) error {
    if err := s.store.UpdateActivityInfo(ctx, a); err != nil {
        return err
    }
    var err error
    for i := 0; i < 3; i++ { // 删除失败重试；仍失败则依赖 TTL 兜底
        if err = s.cache.InvalidateActivity(ctx, a.ID); err == nil {
            return nil
        }
        time.Sleep(time.Duration(i+1) * 50 * time.Millisecond)
    }
    return err
}
```

```go
// Reconcile 对账：Redis 已购人数应等于 MySQL 订单数 + 在途消息数。
// 这里只做告警，修复由人工或补偿任务完成。
func (s *Service) Reconcile(ctx context.Context) {
    list, err := s.store.ListUpcoming(ctx, time.Now())
    if err != nil {
        s.log.Error("reconcile list", "err", err)
        return
    }
    for _, a := range list {
        stock, buyers, err := s.cache.StockSnapshot(ctx, a.ID)
        if err != nil {
            continue
        }
        orders, err := s.store.CountOrders(ctx, a.ID)
        if err != nil {
            continue
        }
        if stock+buyers != a.TotalStock || orders > buyers {
            metrics.ReconcileDiff.Add(1)
            s.log.Warn("reconcile mismatch", "activity", a.ID, "redis_stock", stock,
                "redis_buyers", buyers, "db_orders", orders, "total", a.TotalStock)
        }
    }
}
```

internal/api/handler.go

HTTP 接口、令牌桶限流中间件、错误码映射（4.2 / 4.4.7）。

```go
// Package api 提供 HTTP 接口（Go 1.22+ ServeMux 路由模式）。
package api

import (
    "crypto/subtle"
    "encoding/json"
    "errors"
    "expvar"
    "log/slog"
    "net/http"
    "strconv"

    "golang.org/x/time/rate"

    "github.com/lisa/seckill/internal/metrics"
    "github.com/lisa/seckill/internal/service"
)

// Handler 持有业务服务与限流器。
type Handler struct {
    svc        *service.Service
    limiter    *rate.Limiter // 单实例令牌桶，保护本实例与下游
    adminToken string
    log        *slog.Logger
}

// New 创建 Handler。
func New(svc *service.Service, qps float64, burst int, adminToken string, log *slog.Logger) *Handler {
    return &Handler{svc: svc, limiter: rate.NewLimiter(rate.Limit(qps), burst),
        adminToken: adminToken, log: log}
}

// Routes 注册路由。
func (h *Handler) Routes() http.Handler {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /api/v1/activities/{id}", h.getActivity)
```

```
    mux.Handle("POST /api/v1/activities/{id}/token", h.rateLimit(http.HandlerFunc(h.issueToken)))
    mux.Handle("POST /api/v1/activities/{id}/seckill", h.rateLimit(http.HandlerFunc(h.seckill)))
    mux.HandleFunc("GET /api/v1/activities/{id}/result", h.result)
    mux.HandleFunc("POST /admin/activities/{id}/preheat", h.preheat)
    mux.Handle("GET /debug/vars", expvar.Handler())
    mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    return mux
}

// rateLimit 令牌桶限流中间件，拿不到令牌立即返回 429（不排队等待）。
func (h *Handler) rateLimit(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !h.limiter.Allow() {
            metrics.RateLimited.Add(1)
            writeErr(w, http.StatusTooManyRequests, "server busy, please retry")
            return
        }
        next.ServeHTTP(w, r)
    })
}

func (h *Handler) getActivity(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    a, err := h.svc.Activity(r.Context(), aid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, a)
}

func (h *Handler) issueToken(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    token, err := h.svc.IssueToken(r.Context(), aid, uid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, map[string]string{"token": token})
}

func (h *Handler) seckill(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    var req struct {
        Token string `json:"token"`
    }
    if err := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<10)).Decode(&req); err != nil || req.Token == "" {
        writeErr(w, http.StatusBadRequest, "token required")
        return
    }
```

```
    oid, err := h.svc.Seckill(r.Context(), aid, uid, req.Token)
    if err != nil {
        h.fail(w, err)
        return
    }
    // 202 Accepted：请求已受理，订单异步创建，客户端轮询 /result
    writeJSON(w, http.StatusAccepted, map[string]string{
        "status": "queued", "order_id": strconv.FormatInt(oid, 10)})
}

func (h *Handler) result(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    res, err := h.svc.Result(r.Context(), aid, uid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, res)
}

func (h *Handler) preheat(w http.ResponseWriter, r *http.Request) {
    got := r.Header.Get("X-Admin-Token")
    if subtle.ConstantTimeCompare([]byte(got), []byte(h.adminToken)) != 1 {
        writeErr(w, http.StatusUnauthorized, "unauthorized")
        return
    }
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    if err := h.svc.PreheatByID(r.Context(), aid); err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, map[string]string{"status": "preheated"})
}

// fail 把业务错误映射为 HTTP 状态码；未知错误只记录日志，不向客户端暴露细节。
func (h *Handler) fail(w http.ResponseWriter, err error) {
    switch {
    case errors.Is(err, service.ErrNotFound):
        writeErr(w, http.StatusNotFound, err.Error())
    case errors.Is(err, service.ErrSoldOut), errors.Is(err, service.ErrRepeat):
        writeErr(w, http.StatusConflict, err.Error())
    case errors.Is(err, service.ErrTooMany):
        writeErr(w, http.StatusTooManyRequests, err.Error())
    case errors.Is(err, service.ErrBadToken):
        writeErr(w, http.StatusForbidden, err.Error())
    case errors.Is(err, service.ErrNotStarted), errors.Is(err, service.ErrEnded),
        errors.Is(err, service.ErrNotPreheated):
        writeErr(w, http.StatusBadRequest, err.Error())
    default:
        h.log.Error("internal error", "err", err)
        writeErr(w, http.StatusInternalServerError, "internal error")
    }
}

func pathID(w http.ResponseWriter, r *http.Request) (int64, bool) {
    id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
    if err != nil || id <= 0 {
        writeErr(w, http.StatusBadRequest, "invalid activity id")
        return 0, false
```

```
    }
    return id, true
}

// userID 演示用：从网关注入的 X-User-ID 头读取用户。生产环境应由网关校验 JWT 后注入。
func userID(w http.ResponseWriter, r *http.Request) (int64, bool) {
    uid, err := strconv.ParseInt(r.Header.Get("X-User-ID"), 10, 64)
    if err != nil || uid <= 0 {
        writeErr(w, http.StatusUnauthorized, "login required")
        return 0, false
    }
    return uid, true
}

func writeJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(code)
    _ = json.NewEncoder(w).Encode(v)
}

func writeErr(w http.ResponseWriter, code int, msg string) {
    writeJSON(w, code, map[string]string{"error": msg})
}
```

cmd/seckill/main.go

启动、依赖装配、后台任务、优雅退出（6.2）。

```
// Command seckill 启动秒杀服务。
package main

import (
    "context"
    "errors"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/redis/go-redis/v9"

    "github.com/lisa/seckill/internal/api"
    "github.com/lisa/seckill/internal/cache"
    "github.com/lisa/seckill/internal/config"
    "github.com/lisa/seckill/internal/idgen"
    "github.com/lisa/seckill/internal/service"
    "github.com/lisa/seckill/internal/store"
)

func main() {
    log := slog.New(slog.NewJSONHandler(os.Stdout, nil))
    if err := run(log); err != nil {
        log.Error("exit", "err", err)
        os.Exit(1)
    }
}

func run(log *slog.Logger) error {
    cfg := config.Load()
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    db, err := store.Open(cfg.MySQLDSN)
    if err != nil {
```

```
        return err
    }
    defer db.Close()

    // UniversalClient：Addrs 只有一个地址时为单机客户端，多个地址时为 Cluster 客户端
    rdb := redis.NewUniversalClient(&redis.UniversalOptions{
        Addrs:        []string{cfg.RedisAddr},
        Password:     cfg.RedisPassword,
        DB:           cfg.RedisDB,
        PoolSize:     200,
        DialTimeout:  time.Second,
        ReadTimeout:  500 * time.Millisecond,
        WriteTimeout: 500 * time.Millisecond,
    })
    defer rdb.Close()

    pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel()
    if err := db.Ping(pingCtx); err != nil {
        return err
    }
    if err := rdb.Ping(pingCtx).Err(); err != nil {
        return err
    }

    ids, err := idgen.New(cfg.NodeID)
    if err != nil {
        return err
    }
    svc := service.New(ctx, cfg, cache.New(rdb, cfg.ResultTTL), db, ids, log)

    // 后台任务：预热即将开始的活动、定期对账
    go every(ctx, 30*time.Second, svc.PreheatUpcoming)
    go every(ctx, time.Minute, svc.Reconcile)

    srv := &http.Server{
        Addr:              cfg.HTTPAddr,
        Handler:           api.New(svc, cfg.GlobalQPS, cfg.GlobalBurst, cfg.AdminToken, log).Routes(),
        ReadHeaderTimeout: 2 * time.Second,
        ReadTimeout:       5 * time.Second,
        WriteTimeout:      5 * time.Second,
        IdleTimeout:       60 * time.Second,
    }
    errCh := make(chan error, 1)
    go func() {
        log.Info("listening", "addr", cfg.HTTPAddr)
        if err := srv.ListenAndServe(); !errors.Is(err, http.ErrServerClosed) {
            errCh <- err
        }
        close(errCh)
    }()

    select {
    case err := <-errCh:
        return err
    case <-ctx.Done():
    }
    // 优雅退出：停止接收新请求，等待处理中的请求完成
    shutdownCtx, cancel2 := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel2()
    return srv.Shutdown(shutdownCtx)
}

// every 立即执行一次 fn，之后每隔 d 执行一次，直到 ctx 取消。
func every(ctx context.Context, d time.Duration, fn func(context.Context)) {
    fn(ctx)
    t := time.NewTicker(d)
    defer t.Stop()
    for {
```

```
            select {
            case <-ctx.Done():
                return
            case <-t.C:
                fn(ctx)
            }
        }
    }
```

internal/config/config.go

配置。

```
    // Package config 从环境变量加载配置，给出适合本地演示的默认值。
    package config

    import (
        "os"
        "strconv"
        "time"
    )

    // Config 是服务运行所需的全部配置。
    type Config struct {
        HTTPAddr      string
        MySQLDSN      string
        RedisAddr     string
        RedisPassword string
        RedisDB       int
        AdminToken    string // 管理接口口令，生产环境应接入统一鉴权

        NodeID int64 // 雪花算法机器号，0~1023，多实例部署时必须唯一

        GlobalQPS     float64 // 单实例令牌桶速率
        GlobalBurst   int     // 单实例令牌桶容量
        UserReqPerSec int     // 单用户每秒最多请求数（Redis 固定窗口）

        ConsumerGroup string
        ConsumerName  string
        MaxRetry      int64         // 消息最大投递次数，超过进入死信
        ClaimIdle     time.Duration // pending 消息空闲多久后被重新认领

        TokenTTL  time.Duration // 秒杀令牌有效期
        ResultTTL time.Duration // 抢购结果保留时间
        PreheatIn time.Duration // 提前多久预热活动
    }

    // Load 读取环境变量。
    func Load() Config {
        host, _ := os.Hostname()
        return Config{
            HTTPAddr:      env("HTTP_ADDR", ":8080"),
            MySQLDSN:      env("MYSQL_DSN", "root:root@tcp(127.0.0.1:3306)/seckill?parseTime=true&loc=Local&charset=utf8mb4"),
            RedisAddr:     env("REDIS_ADDR", "127.0.0.1:6379"),
            RedisPassword: env("REDIS_PASSWORD", ""),
            RedisDB:       envInt("REDIS_DB", 0),
            AdminToken:    env("ADMIN_TOKEN", "change-me"),
            NodeID:        int64(envInt("NODE_ID", 1)),
            GlobalQPS:     float64(envInt("GLOBAL_QPS", 5000)),
            GlobalBurst:   envInt("GLOBAL_BURST", 1000),
            UserReqPerSec: envInt("USER_REQ_PER_SEC", 5),
            ConsumerGroup: env("CONSUMER_GROUP", "order-creator"),
            ConsumerName:  env("CONSUMER_NAME", host),
            MaxRetry:      int64(envInt("MAX_RETRY", 5)),
            ClaimIdle:     30 * time.Second,
            TokenTTL:      60 * time.Second,
```

```
        ResultTTL:     30 * time.Minute,
        PreheatIn:     10 * time.Minute,
    }
}

func env(k, def string) string {
    if v := os.Getenv(k); v != "" {
        return v
    }
    return def
}

func envInt(k string, def int) int {
    if v := os.Getenv(k); v != "" {
        if n, err := strconv.Atoi(v); err == nil {
            return n
        }
    }
    return def
}
```

internal/model/model.go

领域模型。

```
// Package model 定义领域模型。
package model

import "time"

// 活动状态。
const (
    ActivityDraft     int8 = 0
    ActivityPublished int8 = 1
    ActivityOffline   int8 = 2
)

// 订单状态。
const (
    OrderUnpaid   int8 = 0
    OrderPaid     int8 = 1
    OrderCanceled int8 = 2
)

// Activity 对应表 seckill_activity。
type Activity struct {
    ID         int64     `json:"id"`
    GoodsID    int64     `json:"goods_id"`
    Title      string    `json:"title"`
    Price      string    `json:"price"` // DECIMAL 以字符串承载，避免浮点误差
    TotalStock int64     `json:"total_stock"`
    Stock      int64     `json:"stock"`
    StartTime  time.Time `json:"start_time"`
    EndTime    time.Time `json:"end_time"`
    Status     int8      `json:"status"`
    Version    int64     `json:"version"`
}

// Order 对应表 seckill_order。
type Order struct {
    ID         int64     `json:"id"`
    ActivityID int64     `json:"activity_id"`
    UserID     int64     `json:"user_id"`
    GoodsID    int64     `json:"goods_id"`
    Price      string    `json:"price"`
    Status     int8      `json:"status"`
```

```
    CreatedAt  time.Time `json:"created_at"`
}
```

internal/idgen/snowflake.go

雪花算法订单号（4.2）。

```
// Package idgen 实现雪花算法：1 位符号 + 41 位毫秒时间戳 + 10 位机器号 + 12 位序列号。
package idgen

import (
    "errors"
    "sync"
    "time"
)

const (
    nodeBits = 10
    seqBits  = 12
    maxNode  = -1 ^ (-1 << nodeBits)
    seqMask  = -1 ^ (-1 << seqBits)
    // epoch 自定义纪元（2024-01-01 UTC），41 位时间戳约可用 69 年。
    epoch = int64(1704067200000)
)

// ErrClockBackwards 表示检测到时钟回拨。
var ErrClockBackwards = errors.New("idgen: clock moved backwards")

// Snowflake 是并发安全的 ID 生成器。
type Snowflake struct {
    mu     sync.Mutex
    node   int64
    lastMs int64
    seq    int64
}

// New 创建生成器，node 取值 0~1023。
func New(node int64) (*Snowflake, error) {
    if node < 0 || node > maxNode {
        return nil, errors.New("idgen: node id out of range")
    }
    return &Snowflake{node: node}, nil
}

// Next 生成下一个 ID。时钟回拨 5ms 以内等待追平，超过则返回错误。
func (s *Snowflake) Next() (int64, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

    now := time.Now().UnixMilli() - epoch
    if now < s.lastMs {
        if s.lastMs-now > 5 {
            return 0, ErrClockBackwards
        }
        for now < s.lastMs {
            time.Sleep(time.Millisecond)
            now = time.Now().UnixMilli() - epoch
        }
    }
    if now == s.lastMs {
        s.seq = (s.seq + 1) & seqMask
        if s.seq == 0 { // 本毫秒序列号用完，等待下一毫秒
            for now <= s.lastMs {
                now = time.Now().UnixMilli() - epoch
            }
        }
```

```
        } else {
            s.seq = 0
        }
        s.lastMs = now
        return now<<(nodeBits+seqBits) | s.node<<seqBits | s.seq, nil
    }
```

internal/metrics/metrics.go

expvar 指标（6.3）。

```
    // Package metrics 用标准库 expvar 暴露计数器（GET /debug/vars），生产可替换为 Prometheus。
    package metrics

    import "expvar"

    var (
        Requests      = expvar.NewInt("seckill_requests_total")
        RateLimited   = expvar.NewInt("seckill_rate_limited_total")
        SoldOut       = expvar.NewInt("seckill_sold_out_total")
        Repeat        = expvar.NewInt("seckill_repeat_total")
        Queued        = expvar.NewInt("seckill_queued_total")
        OrderCreated  = expvar.NewInt("seckill_order_created_total")
        OrderFailed   = expvar.NewInt("seckill_order_failed_total")
        DeadLetters   = expvar.NewInt("seckill_dead_letter_total")
        ReconcileDiff = expvar.NewInt("seckill_reconcile_mismatch_total")
    )
```

go.mod

模块与依赖。

```
module github.com/lisa/seckill

go 1.26.0

require (
    github.com/go-sql-driver/mysql v1.10.1
    github.com/redis/go-redis/v9 v9.23.0
    golang.org/x/sync v0.24.0
    golang.org/x/time v0.16.0
)

require (
    filippo.io/edwards25519 v1.2.0 // indirect
    github.com/cespare/xxhash/v2 v2.3.0 // indirect
    go.uber.org/atomic v1.12.0 // indirect
    golang.org/x/sys v0.48.0 // indirect
)
```